RUANDER
OKTATÓKÖZPONT

# Tesztautomatizálás

Botlik Béla

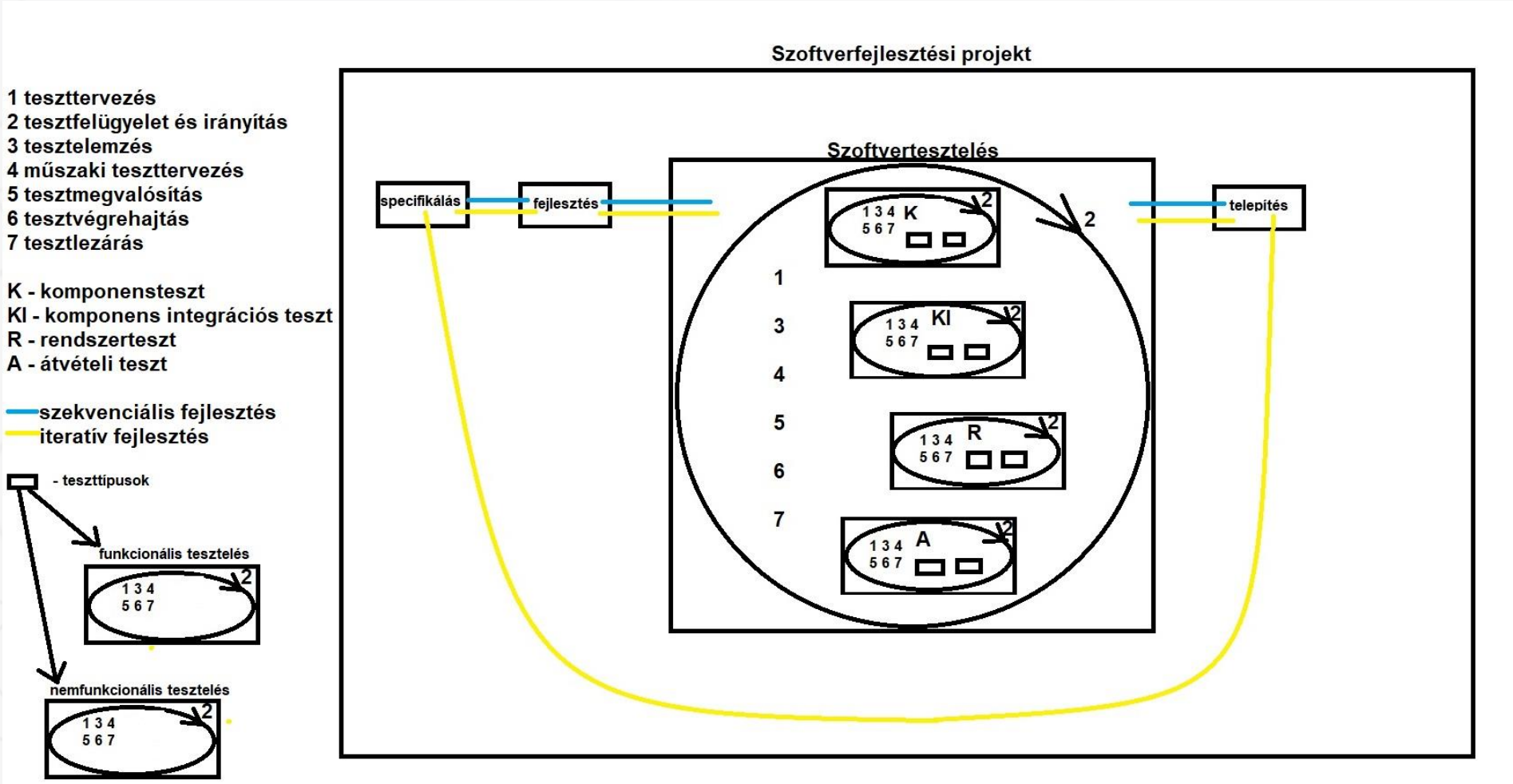
# CTFL ismétlés I.

| Tesztevékenység | Feladatok | Munkatermékek |
|---|---|---|
| Teszttervezés | A teszt céljának, tárgyának, megvalósításának meghatározása. | teszterv, tesztütemterv, kockázati nyilvántartás, belépési és kilépési feltételek |
| Tesztfelügyelet és -irányítás | A tesztevékenységek ellenőrzése és irányítása. | tesztjelentések, irányelvek, kockázati információk, tesztelőrehaladási jelentések |
| Tesztelemzés | A tesztbázis elemzése a szoftver tesztelhetőségének feltárására. | tesztfeltételek, tesztbázis hibajelentései |
| Műszaki teszttervezés | Tesztesetek készítése, tesztkörnyezet tervezése. | tesztesetek, lefedettségi elemek, tesztadatokra és a tesztkörnyezetre vonatkozó követelmények |
| Tesztmegvalósítás | Teszteljárások, tesztkészletek készítése, tesztkörnyezet megvalósítása. | teszteljárások, tesztkészletek, tesztszkriptek, tesztadatok, végrehajtási ütemterv, tesztkörnyezet |
| Tesztvégrehajtás | A tesztek futtatása a tesztelési ütemterv szerint. | tesztnaplók, hibajelentések |
| Tesztlezárás | Tesztjelentések készítése. | összefoglaló tesztjelentés, változtatáskérés |

# CTFL ismétlés II.

| | tesztbázis | tesztcél | teszt tárgya | hibák | megközelítés |
|---|---|---|---|---|---|
| **komponensteszt** | tervek, kód, adatmodell | komp. verifikálás, hibák feltárása, bizalom kivívása | komponens, osztály, adatstruktúra | hibás funkcionalitás, programozói hibák | Kódközeli, tipikusan a fejlesztők végzik. |
| **komponensintegrációs teszt** | tervezési diagramok, interfész leírások | kockázat csökkentés, interfész verifikálás, hibák feltárása | interfészek | helytelen/hiányos adatok, hívások helytelen sorrendje, adatok félreértelmezése | Funkcionalitás helyett a kommunikációra fókuszál, a fejlesztők végzik. |
| **rendszerteszt** | rendszer/szoftver-követelmények | rendszer verifikálás, bizalom kivívása | alkalmazás, op.rendszer, rendszer | helytelen viselkedés, helytelen vezérlés | Végponttól végpontig terjedő tesztek, független tesztelők is végezhetik. |
| **rendszerintegrációs teszt** | rendszerterv, használati esetek | kockázat csökkentés, interfész verifikálás, hibák feltárása | alrendszerek, API-k, külső szolgáltatások | rendszerek közötti kommunikáció hibái, átadott adatok félreértelmezése | Funkcionalitás helyett a kommunikációra fókuszál, a tesztelők végzik. |
| **elfogadási teszt** | rendszer/szoftver-követelmények | rendszer validálás, kiadásra való felkészültség igazolása | rendszer | követelményeknek nem megfelelés | A tesztek felelősei üzleti felhasználók, vevők. |

# CTFL ismétlés III.



# Bevezetés és célok a tesztautomatizálás terén (I. fejezet)

- A tesztautomatizálás célja
- A tesztautomatizálás előnyei
- A tesztautomatizálás hátrányai
- A tesztautomatizálás korlátai
- Tesztautomatizálás különböző fejlesztési életciklusmodellekben
- Teszteszköz választás

# Tesztautomatizálás

- A tesztautomatizálás tevékenységei:
  - célzott szoftvereszközök használata tesztkészletek előkészítéséhez és kezeléséhez a tesztvégrehajtás érdekében;
  - tesztek automatizált végrehajtása;
  - az aktuális és elvárt eredmények összehasonlítása.
- A tesztautomatizálás széleskörű szoftverterületeket fed le és sokféle szoftverre vonatkozhat
  - UI SUT; SUT UI nélkül; mobilappok; hálózati protokollok és kapcsolatok

# A tesztautomatizálás előnyei, hátrányai és korlátai

| Előnyök | Hátrányok | Korlátok |
|---|---|---|
| Több teszt futtatása | Kezdeti befektetés szükséges | Nem minden teszt automatizálható |
| Manuálisan végre nem hajtható tesztek futtatása | A tesztautomatizálási megoldás fejlesztése és karbantartása időigényes lehet | Csak azt tudja tesztelni, amire programozták |
| Összetettebb tesztek elvégzése | Esetleges plusz költségek | Teszt-orákulum hitelesítést igényel |
| Tesztelési erőforrások hatékony kihasználása | Kevésbé alkalmazkodó a rendszer változásaihoz | Csak géppel értelmezhető teszteredményeket tud ellenőrizni |
| Kevesebb emberi hiba | Új hibákat vezethet be | |
| Gyorsabb végrehajtás | Világos tesztcélokra van szükség | |
| Gyors visszajelzés a minőségről | | |
| Javítja a rendszer megbízhatóságát | | |
| Következetes tesztvégrehajtás | | |

# Tesztautomatizálás és fejlesztési modellek

- A különböző fejlesztési modellek más jellemzőkkel bírnak a tesztautomatizálás szempontjából:
  - Vízesés modell: a tesztautomatizálás a megvalósítással párhuzamosan vagy azt követően zajlik.
  - V-modell: minden fejlesztési szinthez tartozik tesztelési szint.
  - Agilis fejlesztés: gyakori automatizált tesztvégrehajtás jellemzi.
- A legmegfelelőbb teszteszköz kiválasztásához a tesztelt rendszert (SUT) és a csapat összetételét kell kiértékelni.

# Felkészülés a tesztautomatizálásra (II. fejezet)

- Tesztelhetőség és konfiguráció
- Tesztautomatizálás végrehajtási környezetei
- A tesztelendő rendszer elemzése
- Eszközértékelés

# Tesztelhetőség és konfigurációs megoldások

- Az automatizálásra való felkészülés első lépése a tesztelhetőség meghatározása.
- A jobb tesztelhetőség konfigurációs beállításokat igénylő megoldásai:
  - hozzáférhetőségi azonosítók;
  - rendszerkörnyezeti változók;
  - telepítési változók.
- A tesztelhetőség tervezésének aspektusai:
  - megfigyelhetőség;
  - irányíthatóság;
  - architektúra átláthatósága.

# Tesztautomatizálás különböző környezetben

- Automatizált tesztek végrehajtási környezetei:
  - lokális fejlesztési környezet: komponensteszt; fehérdobozos teszt
  - build környezet: CI/CD folyamat; komponensintegrációs teszt
  - integrációs környezet: feketedobozos és rendszer(integrációs) teszt
  - validációs környezet: elfogadási tesztelés
  - működési környezet: speciális tesztek (canary release, blue/green deployment, A/B teszt)

# A tesztelendő rendszer elemzése

- Egy sikeres tesztautomatizálási megoldás (TAS) létrehozásának előfeltétele a tesztelendő rendszer (SUT) elemzése.
- A tesztautomatizálási megközelítés és architektúra követelményeinek szempontjai:
  - tesztttevékenységek;
  - tesztszintek;
  - teszttípusok;
  - tesztszerepkörök;
  - szoftvertermékek;
  - SUT és TAS kompatibilitás;
  - tesztadatok elérhetősége és minősége;
  - harmadik féltől származó alkalmazások emulálása.

# Feladat I.

- Elemezze a tesztelt rendszert (SUT) és keressen tesztautomatizálási megoldást!
  - A vizsgált rendszer egy webáruház, amely lehetővé teszi a felhasználók számára:
    - Regisztrációt és bejelentkezést
    - Termékek keresését és megtekintését
    - Kosárba helyezést és vásárlást
    - Fizetést PayPal integrációval
  - A rendszer többféle technológiai rétegből áll:
    - Frontend (React)
    - Backend (Java – Spring Boot)
    - Adatbázis (MySql)
    - Külső API (PayPal)

# Az eszközértékelés eredményei

- A rendszer elemzéséből táblázatos formában összeáll a használható eszközök köre, legfőbb tulajdonságaikkal:
  - technológia és IDE;
  - konfigurálhatóság;
  - tesztadatok kezelhetősége;
  - teszttípusonkénti eltérések;
  - riportálási képesség;
  - más eszközökkel való integrálhatóság;
  - az architektúra bővítésének lehetősége.

# Feladat II.

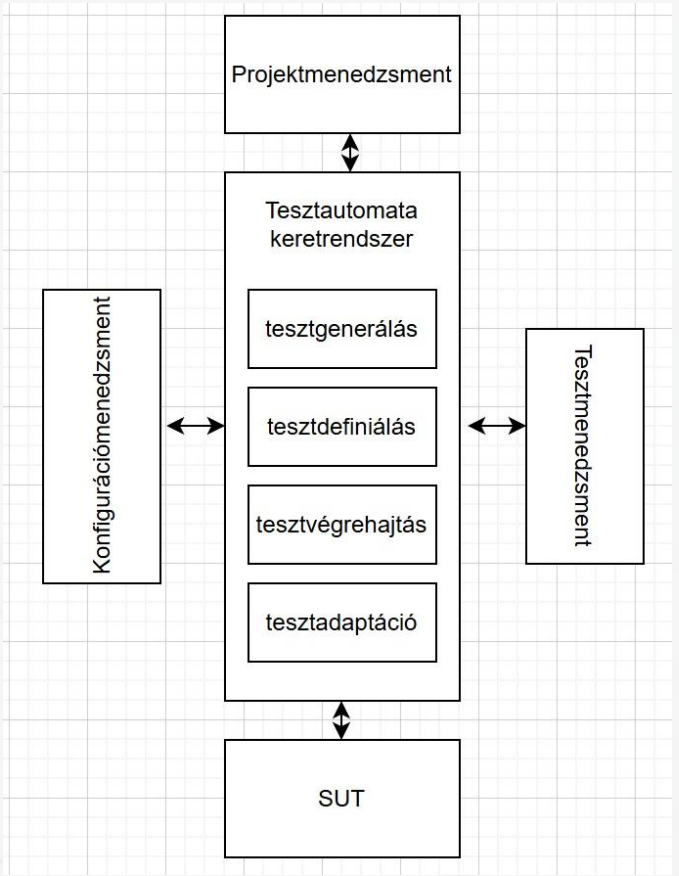
- Az előző feladat webáruházának PayPal API teszteléséhez első körben 3 eszköz merült fel:
  - Postman - Grafikus felhasználói felülettel rendelkező API tesztelő
  - RestAssured - Java alapú API tesztelési könyvtár fejlesztőközpontú megközelítéssel
  - Karate - BDD megközelítésű API tesztelő keretrendszer
- Töltse ki az eszközértékelés eredményeit összehasonlító táblázatot!

# Tesztautomatizálási architektúra (III. fejezet)

- Általános tesztautomatizálási architektúra
- Tesztautomatizálási megoldás tervezése
- Tesztautomatizálási keretrendszer (TAF)
- Tesztautomatizálási megközelítések
- Tervezési alapelvek és tervezési minták a tesztautomatizálásban

# A tesztautomatizálási architektúra és fő képességei

- A tesztautomatizálási architektúra interfészei:
  - SUT interfész;
  - Projektmenedzsment interfész;
  - Tesztmenedzsment interfész;
  - Konfigurációmenedzsment interfész.
- Az automatizálási keretrendszer képességei:
  - Tesztgenerálás;
  - Tesztdefiniálás;
  - Tesztvégrehajtás;
  - Tesztadaptáció.



# Tesztautomatizálási megoldás tervezése

- A tesztautomatizálási megoldás (TAS) tervezése a SUT követelményei és a szükséges eszközök alapján kerül megvalósításra, melynek elemei:
  - Tesztautomata eszközök és eszközspecifikus könyvtárak kiválasztása
  - Beépülő modulok és/vagy komponensek fejlesztése
  - Kapcsolati és interfész követelmények azonosítása
  - Kapcsolódás tesztmenedzsment és hibakezelő eszközökhöz
  - Verziókezelési rendszer és adattárak használata

# Tesztautomatizálási keretrendszer

- A TAS technikai alapja a tesztautomatizálási keretrendszer (TAF).

| Elemei | Jellemzői | Rétegei |
|---|---|---|
| tesztellenőrző rendszer | struktúráltság | tesztszkriptek |
| tesztkönyvtárak | automatizálási hatékonyság | üzleti logika |
| tesztszkriptek | skálázhatóság | alapkódkönyvtárak |
| tesztkészletek | | |

# Feladat III.

- Alkalmazza a tesztautomatizálási keretrendszer rétegezését egy webalkalmazás funkcionális teszteléséhez (belépési funkció)!

# Tesztautomatizálási megközelítések I.

- Számos megközelítés létezik a tesztautomatizálás terén, melyek különböző körülmények között alkalmazhatók:

| Megközelítés | Főbb jellemzők | Előnyök | Hátrányok |
|---|---|---|---|
| Rögzítés/lejátszás | A manuális tesztelési műveletek elektronikus rögzítésén és visszajátszásán alapul. | Kezdetben könnyen beállítható és használható. | Fenntartása/továbbfejlesztése nehézkes;<br>SUT elérhető kell legyen;<br>Ritkán és kismértékben változó SUT esetén ésszerű;<br>SUT-verziófüggő;<br>A tesztesetek rögzítése időigényes lehet. |
| Lineáris szkriptelés | A rögzítés/lejátszás tesztjeit felhasználó programozási tevékenység | Gyors és egyszerű;<br>A tesztszkriptek módosíthatók. | Fenntartása/továbbfejlesztése nehézkes;<br>SUT elérhető kell legyen;<br>Ritkán és kismértékben változó SUT esetén ésszerű;<br>Némi programozási ismeret szükséges. |

# Tesztautomatizálási megközelítések II.

| Megközelítés | Főbb jellemzők | Előnyök | Hátrányok |
|---|---|---|---|
| Struktúrált szkriptelés | Blokkokba, könyvtárakba szervezett szkriptek | újrafelhasználható;<br>karbantartható;<br>rugalmas;<br>az üzleti logika elkülöníthető. | programozási ismeret szükséges;<br>nagyobb kezdeti befektetés;<br>komplexitás. |
| Adatalapú tesztelés (DDT) | Struktúrált szkriptre épülő, adatokkal ellátott szkriptek | tesztesetek egyszerű bővítése;<br>tesztköltségek csökkenése;<br>tesztelemzők nagyfokú szabadsága. | adatkezelési kihívások |
| Kulcsszóvezérelt tesztelés (KDT) | Gyakran DDT-re épülő, kulcsszavak és tesztadatok alapján származtatott lépések listája vagy táblázata | teszt -és üzleti elemzők részvétele a tesztesetek létrehozásában;<br>manuális tesztekre is alkalmazható;<br>kevésbé függ a fejlesztktől. | A kulcsszavak implementálása és karbantartása összetett feladat;<br>Kis rendszerek esetén jelentős erőforrás igénye van. |

# Tesztautomatizálási megközelítések III.

| Megközelítés | Főbb jellemzők | Előnyök | Hátrányok |
|---|---|---|---|
| TDD (tesztvezérelt fejlesztés) | A tesztek a fejlesztés részeként és elsőként kerülnek meghatározásra | egyszerű teszteset fejlesztés;<br>jobb kódminőség, - tesztelhetőség, - kódlefedettség elérés;<br>csökkenti a hibák tovább terjedését;<br>javuló kommunikáció;<br>gyors kilépési kritérium elérés. | TDD-re áttérés időigényes; |
| BDD (viselkedésvezérelt fejlesztés) | Természetes nyelvformátum használatával kialakított elfogadási kritériumok, amik egyúttal tesztesetek is | javuló kommunikáció;<br>BDD forgatókönyvek tesztesetként biztosítják a specifikáció lefedettségét;<br>többféle teszttípus előállításában használható a tesztpiramis különböző szintjein. | további teszteseteket kell meghatározni;<br>üzleti elemzők és fejlesztők kihagyása;<br>tesztlépések implementálása bonyolult;<br>emiatt nehézkes és költséges hibakeresés. |

# Feladat IV.

- Határozzon meg különböző tesztautomatizálási megközelítéseket egy e-kereskedelmi webáruház kosár és fizetés folyamat teszteléséhez!

# Tervezési alapelvek és tervezési minták a tesztautomatizálásban

- A szoftverfejlesztő mellett a tesztautomatizálási mérnöknek is ismernie kell a tervezési alapelveket és mintákat:

| OOP alapelvek | SOLID alapelvek | Tervezési minták |
|---|---|---|
| absztrakció | egyért felelősség | Facade pattern |
| egységbezárás | nyitottság és zártság | Singleton pattern |
| adatrejtés | Liskov helyettesítés | POM (Page Object Model) |
| öröklődés | interfész helyettesítés | Folyamat-alapú modell (FMP) |
| polimorfizmus | függőség inverzió | |

# Feladat V.

- Adjon meg néhány tervezési mintát -a tervezési alapelvek betartásával- egy e-kereskedelmi webáruház termék kereső és kosárba helyezés folyamatához!

# A tesztautomatizálás megvalósítása (IV. fejezet)

- Pilot projekt
- Folyamat integráció és szervezeti tényezők
- Kockázatelemzés és kockázatcsökkentés a tesztautomatizálásban
- A tesztautomatizálási megoldás fenntarthatósága

# A tesztautomatizálási pilot projekt sikerének kulcsténye zői

- A pilot projekt egy rövid, de kulcsfontosságú tesztidőszak a tesztautomatizálási stratégia meghatározásához.
- Az értékelés legfontosabb tényezői:
  - Használt programozási nyelvek és eszközök (kereskedelmi vagy nyílt forráskódú)
  - Lefedendő tesztszintek
  - Kiválasztott tesztesetek
  - Tesztesetek fejlesztési megközelítése

# CI/CD integráció és szervezeti tényezők a tesztautomatizálásban

- CI/CD folyamat integrációja:
  - A tesztautomatizálási rendszer (TAS) beillesztése a folyamatos integrációs folyamatokba.
  - A kezdeti CI/CD beállítás módosítása a tesztek növekedésével.
- Egyéb, nem technikai szempontok értékelése:
  - A csapattagok tudása és tapasztalata
  - A csapat szerkezete és együttműködése
  - Licencelési és szervezeti szabályok
  - A tesztelési célok és tesztesetek lefedése

# Feladat VI.

- Egy cég úgy döntött, bevezeti a tesztautomatizálást a weboldaluk fizetési folyamatának ellenőrzésére, amit pilot projekttel indítanak. Ehhez Selenium Webdrivert választanak.
  - Alkalmazzon irányelveket, amelyek támogatják a hatékony tesztautomatizálási pilot és bevezetési tevékenységeket!

# Telepítési kockázatok elemzése

- TAF és SUT összekapcsolása
- Bevezetési kockázatok
  - Tűzfalak megnyitása
  - Erőforrások kihasználtsága (CPU, RAM)
- Felkészülés és kockázatkezelés
  - Hálózati kapcsolat és megbízhatóság
  - Mobil tesztautomatizálás kihívásai

# Tesztautomatizálás és kockázatcsökkentés

- Technikai bevezetési kockázatok
  - Tesztcsomag összeállítása
  - Naplózás és logolási szintek
  - Tesztstruktúra kialakítása
  - Frissítések kezelése
- Kockázatcsökkentési stratégiák
  - Verziókezelés és központi tárolás
  - Megbízható tesztkörnyezet és előfeltételek
  - Tesztek ismételhetősége és automatizáció

# Feladat VII.

- Egy fejlesztőcsapat automatizált regressziós tesztelést szeretne bevezetni egy banki mobilalkalmazásban. A cél az, hogy minden új frissítés után ellenőrizzék az alapvető funkciók működését.
- Elemezze a bevezetési kockázatokat, és tervezze meg az enyhítési stratégiákat!

# A tesztautomatizálási megoldás fenntarthatósága

- A karbantarthatóság érdekében ajánlott a tiszta kód elveinek követése (Robert C. Martin: Clean Code):
  - egységes elnevezési konvenciók használata;
  - hardcode kerülése;
  - bemeneti paraméterek minimalizálása;
  - hosszú és bonyolult metódusok kerülése;
  - naplózás használata;
  - tervezési minták alkalmazása;
  - tesztelhetőség biztosítása.